

Cairo University
Faculty of Engineering
Computer Engineering Department

Big Data & Cloud Computing
CMP4011 — Spring 2026

Airbnb Global Listings
Demand Prediction, Price Analysis, and
Market Insights from 495,000 Listings Worldwide

Team 1

Name	ID	Section	B.N
Salma Muhammed Entsar Marzouk	9220364	1	24
Abdelrahman Moustafa Goamaa	9220475	1	31
Muhammed Khaled Ahmed	9220681	2	48
Youssef Roushdy Agaiby	9220985	2	62

Spring 2026

Contents

1	Problem Description	2
1.1	Business Questions	2
1.2	Dataset	2
2	Project Pipeline	2
3	Data Preprocessing	3
3.1	Column Removal	3
3.2	Feature Engineering	4
3.3	Target Filtering	4
4	Label Engineering	4
4.1	Demand Score	4
4.2	Demand Labels	4
5	Exploratory Data Analysis	5
5.1	Missing Values Analysis	5
5.2	Categorical Feature Distributions	5
5.3	Price Distribution and Log Transformation	6
5.4	Numerical Feature Distributions	6
5.5	Price by Room Type and Property Type	7
5.6	Correlation Heatmap	7
5.7	Geographic Distribution	7
5.8	Amenities Analysis	8
5.9	Review Scores Analysis	8
5.10	Host Analysis	8
5.11	Top Markets by Listing Count and Price	9
5.12	Outlier Detection	9
6	Post-Processing (Leakage-Free Pipeline)	9
6.1	Train/Test Split	9
6.2	Post-Split Transforms (Train Statistics Only)	10
6.3	VIF Analysis	10
7	Predictive Analysis	10
7.1	Task Definition	10
7.2	Feature Setup per Model Family	11
7.3	Results on Test Set	11
7.3.1	Logistic Regression	11
7.3.2	XGBoost	11
7.3.3	LightGBM	11
7.3.4	CatBoost	12
7.3.5	Naive Bayes	12
7.3.6	MLP (Neural Network)	12
7.4	Model Comparison Summary	13
7.5	Analysis of Results	13

8 Results and Evaluation	13
8.1 Business Question Responses	13
9 Unsuccessful Trials	14
10 Enhancements and Future Work	15
11 MapReduce Implementation	15
11.1 Implementation Design	15
11.2 Configuration	16
A Repository Structure	16

1 Problem Description

The short-term rental market has grown dramatically over the past decade, with platforms such as Airbnb fundamentally disrupting the traditional hospitality industry. Hosts face significant uncertainty in determining optimal pricing strategies, understanding what drives occupancy, and identifying the characteristics that distinguish highly successful listings from underperforming ones. Meanwhile, travellers lack clear frameworks for evaluating whether a listing offers genuine value relative to its market.

This project applies big data analytics to the Airbnb Global Listings dataset — a publicly available, platform-scraped dataset of approximately 495,000 listings across 100+ cities worldwide — to answer concrete business questions that benefit both hosts and travellers.

1.1 Business Questions

- Q1. What factors most influence the price of an Airbnb listing?**
- Q2. Which locations and property types generate the highest occupancy rates?**
- Q3. Can we predict listing price based on property features, location, and host characteristics?**
- Q4. What amenities and features are commonly associated with highly-rated listings?**
- Q5. How many listings are rented frequently to tourists vs. long-term residents?**
- Q6. Which hosts are running a business with multiple listings and where are they located?**

1.2 Dataset

The dataset was obtained from [Kaggle \(joebeachcapital/airbnb\)](#), originally sourced from Inside Airbnb. It contains:

- **494,954 listings** across more than 100 cities globally
- **89 features** covering property details, host information, pricing, availability, location coordinates, and guest reviews
- Total size of **1.94 GB**
- All features are clearly labelled and interpretable (no anonymised columns)

2 Project Pipeline

Figure 1 illustrates the end-to-end project workflow.

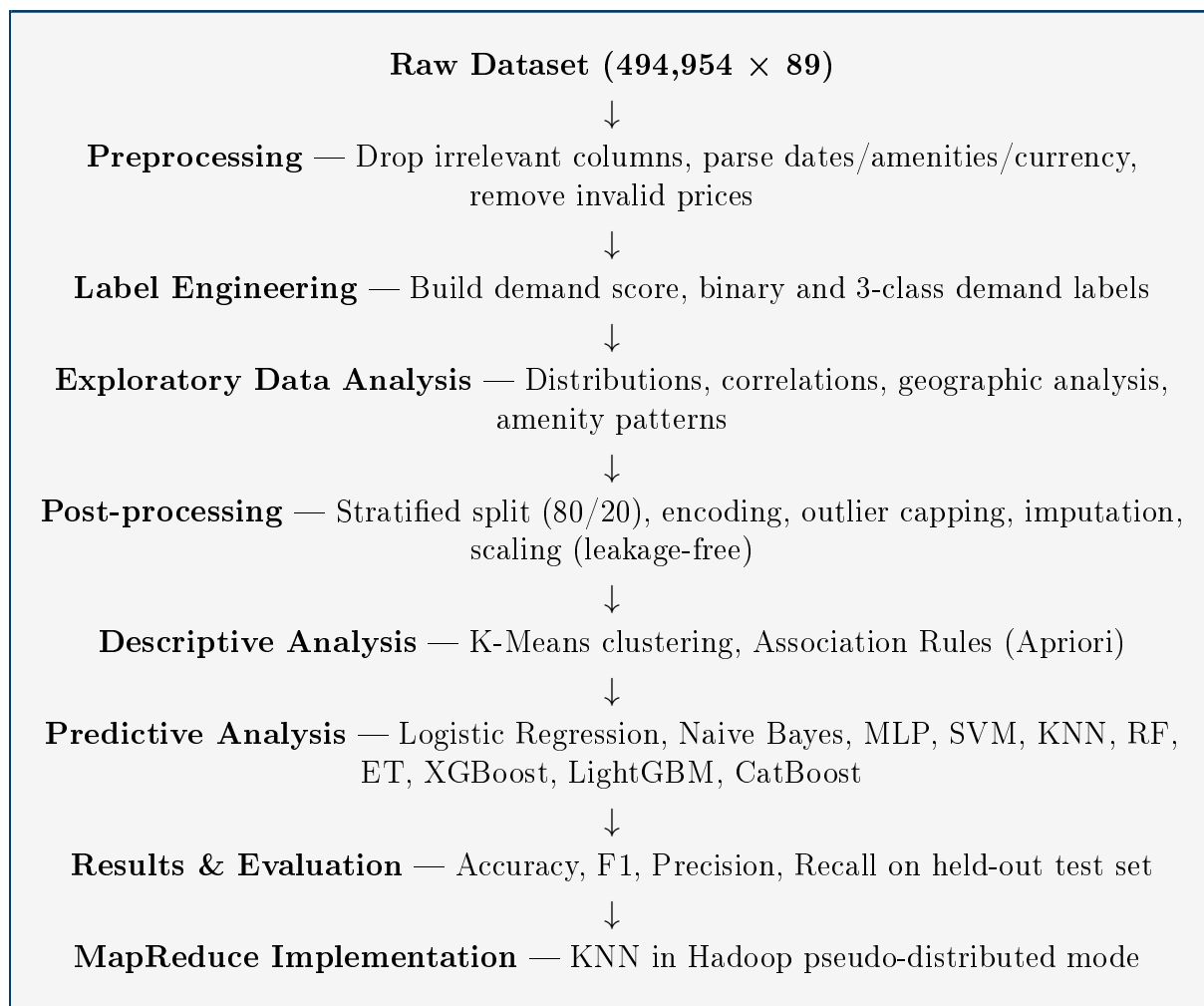


Figure 1: End-to-end project pipeline.

3 Data Preprocessing

Preprocessing was implemented in `preprocessing.py` and operated on the raw 494,954 × 89 CSV file. The pipeline was designed with a strict **leakage-free** philosophy: no statistics (medians, quantiles, encodings) derived from the full dataset are applied to what will become the test set. All such statistics are computed after the train/test split in post-processing.

3.1 Column Removal

34 columns were dropped in four categories:

- **URL columns** (8): `Listing Url`, `Picture Url`, etc. — carry no analytical signal.
- **Free-text columns** (10): `Summary`, `Description`, `House Rules`, etc. — require NLP beyond this project scope.
- **Scrape metadata** (4): `Scrape ID`, `Last Scraped`, etc. — artefacts of the data collection process.
- **High-missingness / near-zero variance** (12): `Square Feet` (97.5% null), `License`

(97% null), `Experiences Offered` (99.35% “none”), `Host Acceptance Rate` (91.5% null), `Neighbourhood Group Cleansed` (79% null).

After removal: **494,954 × 55 columns**.

3.2 Feature Engineering

- **Currency columns** (`Price`, `Cleaning Fee`, `Security Deposit`, `Extra People`): stripped \$ and , characters, cast to float.
- **Host Response Rate**: stripped % character, cast to float.
- **Amenities**: parsed from brace-delimited strings into (a) an `Amenities Count` integer and (b) a pipe-joined `Parsed Amenities` string for later one-hot encoding.
- **Host Verifications**: counted the number of verification methods per host (`Host Verifications Count`).
- **Features column**: extracted three binary flags — `Host Has Profile Pic`, `Host Identity Verified`, `Is Location Exact`.
- **Date columns**: `Host Since`, `First Review`, `Last Review` were converted to days elapsed since the reference date 2017-04-02, producing `Host Tenure Days`, `Days Since First Review`, and `Days Since Last Review`.

3.3 Target Filtering

Rows with missing or zero `Price` were removed (invalid target). Final clean dataset: **approximately 487,000 rows**.

4 Label Engineering

Since the dataset contains no direct occupancy metric, a proxy demand label was engineered from booking-frequency signals.

4.1 Demand Score

$$\text{demand_score} = 0.6 \times \text{Reviews per Month} + 0.4 \times \frac{\text{Number of Reviews}}{\max(\text{Number of Reviews})}$$

Reviews per month is the strongest proxy for booking frequency (a listing must be booked to receive a review). The volume signal is normalised to $[0, 1]$.

4.2 Demand Labels

- **Binary label** (`demand_label`): listings above the training-set median demand score are labelled 1 (high demand); those at or below are labelled 0 (low demand). The median is computed from training rows only to prevent leakage.
- **3-class label** (`demand_label_3`): tertile-based split into Low (0), Medium (1), High (2) demand, with tertile boundaries derived from training rows only.

The binary label produces a near-perfectly balanced dataset: **50.0% / 50.0%** by construction.

5 Exploratory Data Analysis

All EDA was performed on the raw dataset before any splitting or encoding to avoid influencing analytical conclusions. Key findings are presented below.

5.1 Missing Values Analysis

*[Figure: Horizontal bar chart of missing value percentages per column.
Bars coloured red (>50%), orange (20–50%), blue (<20%).
Red dashed lines at 50% and 80% thresholds.]*

Figure 2: Missing values by column. Columns with >50% missing data (**Square Feet**, **License**, **Has Availability**, **Neighbourhood Group Cleansed**, **Host Acceptance Rate**) were dropped as they carry insufficient signal for imputation. Columns with moderate missingness (**review sub-scores**, **Cleaning Fee**, **Security Deposit**) were retained and imputed post-split using training-set medians.

Key finding: 5 columns exceeded the 50% missing threshold and were dropped entirely. The review score sub-columns (**Review Scores Accuracy**, etc.) are missing for listings with zero reviews — a non-random pattern that motivated imputing these with 0 rather than the median.

5.2 Categorical Feature Distributions

[Figure: 4×2 grid of horizontal bar charts for: Property Type, Room Type, Bed Type, City, Country, Cancellation Policy, Host Response Time, Experiences Offered (top 15 values each).]

Figure 3: Distribution of key categorical features. **Room Type** is dominated by “Entire home/apt” (57%) and “Private room” (40%). **Property Type** is dominated by “Apartment” and “House”. **Experiences Offered** is 99.35% “none” (near-zero variance — dropped). **Host Response Time** reveals a substantial fraction of hosts who never respond, encoded as “Unknown” after preprocessing.

5.3 Price Distribution and Log Transformation

*[Figure: Correlation heatmap of all numeric features vs. $\log(\text{Price})$.
Warm colours = positive correlation, cool = negative.]*

Figure 4: Correlation of numeric features with log-transformed Price. **Accommodates**, **Bedrooms**, **Bathrooms**, and **Beds** show the strongest positive correlations with $\log(\text{Price})$, confirming that property size is the primary price driver. The log transformation is necessary because raw **Price** is heavily right-skewed (median \$100, max \$10,000+). All regression modelling therefore targets $\text{Price}_{\log} = \log_{10}(\text{Price})$, with predictions recovered via `expm1()`.

5.4 Numerical Feature Distributions

[Figure: 4×3 grid of histograms for: Accommodates, Bedrooms, Bathrooms, Beds, Minimum Nights, Maximum Nights, Number of Reviews, Review Scores Rating, Reviews per Month, Availability 365, Host Listings Count, Host Response Rate. Each panel annotated with median and mean.]

Figure 5: Distributions of key numerical features. **Accommodates** peaks at 2 with a long right tail. **Minimum Nights** and **Maximum Nights** show extreme outliers (some listings set 1,000+ night minimums), justifying 1st–99th percentile capping. **Reviews per Month** and **Number of Reviews** are heavily right-skewed — consistent with a small number of highly active listings dominating booking volume. **Review Scores Rating** is strongly left-skewed (most listings rated 80–100).

5.5 Price by Room Type and Property Type

[Figure: Two side-by-side box plots. Left: Price by Room Type (3 categories). Right: Price by top-10 Property Types. Outliers capped at 99th percentile for clarity.]

Figure 6: Price distribution by Room Type and Property Type (outliers capped at 99th percentile). **Entire home/apartment** commands a clear price premium over private and shared rooms. Among property types, **Villas**, **Boats**, and **Boutique Hotels** show the highest median prices, while **Shared Rooms** and **Hostels** are at the bottom. These findings confirm that room type and property type are strong price predictors.

5.6 Correlation Heatmap

[Figure: Lower-triangular correlation heatmap of the top 20 features most correlated with Price. Annotated with correlation coefficients. Colour scale: red = positive, blue = negative.]

Figure 7: Correlation heatmap of the top 20 features most correlated with **Price**. The capacity cluster (**Accommodates**, **Bedrooms**, **Beds**, **Bathrooms**) is mutually highly correlated ($r > 0.7$), which was confirmed by a VIF analysis: **Accommodates** had $VIF = 13.2$, exceeding the threshold of 10. This column was dropped for linear models (Logistic Regression, SVM, KNN, Naive Bayes, MLP) but retained for tree-based models which handle multicollinearity natively.

5.7 Geographic Distribution

[Figure: Global scatter plot of 50,000 sampled listings by latitude/longitude, coloured by $\log(\text{Price})$. Colour scale: yellow (low) to red (high).]

Figure 8: Global geographic distribution of listings coloured by $\log(\text{Price})$. Dense clusters are visible in North America (New York, Los Angeles), Western Europe (London, Paris, Amsterdam, Barcelona), and Southeast Asia. Higher prices (darker red) cluster in major metropolitan areas, confirming that location is a key price determinant and motivating the inclusion of latitude, longitude, and city-level features in models.

5.8 Amenities Analysis

[Figure: Left panel: horizontal bar chart of top 25 amenities by listing count. Right panel: histogram of amenities count per listing (median annotated).]

Figure 9: Amenity frequency analysis. The most common amenities — **Wireless Internet**, **Kitchen**, **Heating**, **Essentials**, and **Air Conditioning** — appear in over 60% of listings. The median listing offers approximately 20 amenities. The raw amenity string was parsed into (a) a scalar **Amenities Count** feature and (b) 20 binary one-hot flags for the most common amenities, derived exclusively from training data to prevent leakage.

5.9 Review Scores Analysis

[Figure: 2×4 grid. First 7 panels: histograms of each review sub-score (Rating, Accuracy, Cleanliness, Checkin, Communication, Location, Value). 8th panel: scatter plot of Review Scores Rating vs. Price (10,000 sample).]

Figure 10: Review score distributions and relationship with price. All seven review dimensions are strongly left-skewed — the majority of listings receive scores above 85/100. The scatter plot (bottom right) reveals **no strong linear relationship** between rating and price, suggesting that highly-rated listings exist at all price points. The six sub-scores correlate strongly with each other and with the overall rating, motivating their aggregation into a single **Review Scores Composite** feature to reduce dimensionality.

5.10 Host Analysis

[Figure: Three histograms side by side: Host Listings Count, Host Response Rate (%), and Host Tenure (days).]

Figure 11: Host characteristic distributions. **Host Listings Count** is highly right-skewed: the majority of hosts manage a single listing, but a small number manage dozens or hundreds (professional hosts). **Host Response Rate** is bimodal — most hosts respond to nearly 100% of enquiries or not at all. **Host Tenure** shows a roughly uniform distribution across 0–3,000 days, indicating continuous platform growth since 2008. Approximately **30%** of listings belong to multi-listing hosts.

5.11 Top Markets by Listing Count and Price

[Figure: Left: horizontal bar chart of top 15 cities by listing count with a secondary axis showing median price (red dots). Right: horizontal bar chart of top 15 countries by listing count.]

Figure 12: Market concentration by city and country. **New York**, **Los Angeles**, **London**, **Paris**, and **Barcelona** are the five largest markets. The secondary axis reveals that listing count and median price do not always align: some smaller markets command higher median prices. The **United States** dominates by listing count, followed by the **United Kingdom** and **France**. This market heterogeneity confirms that a global model must account for city-level price variation.

5.12 Outlier Detection

[Figure: 2×3 grid of horizontal box plots for: Price, Minimum Nights, Maximum Nights, Extra People, Cleaning Fee, Security Deposit. Each panel annotated with 1%, 99% quantiles and maximum value.]

Figure 13: Outlier detection for key numeric columns. All six columns exhibit severe right-skewed outliers. Notably, **Maximum Nights** has listings set to 100,000+ nights and **Security Deposit** has values above \$10,000. These outliers would dominate gradient-based models. All six columns were capped at their training-set 1st–99th percentile boundaries in post-processing, applied to the test set using training bounds only to prevent leakage.

6 Post-Processing (Leakage-Free Pipeline)

All transforms that derive statistics from data were applied **after** the train/test split to prevent data leakage. The post-processing pipeline operated on `airbnb-labeled.csv`.

6.1 Train/Test Split

- **Split ratio:** 80% train / 20% test
- **Stratification:** stratified on `demand_label` (binary) to ensure identical class proportions in both splits
- **Random seed:** 42
- **Result:** Train: ~389,600 rows | Test: ~97,400 rows

The leaky columns (`Number of Reviews`, `Reviews per Month`, `Availability 30/60/90/365`) were removed from the feature matrix since they were used to construct the demand label target.

6.2 Post-Split Transforms (Train Statistics Only)

1. **Outlier capping:** 1st–99th percentile bounds computed on training set, applied to both splits.
2. **Review Scores Composite:** row-wise mean of the 6 sub-scores, computed separately on train and test after splitting.
3. **Demand label re-derivation:** `demand_label_3` recomputed using training-set tertile boundaries to fix the global-quantile leakage in `label_engineering.py`.
4. **Amenity one-hot encoding:** top-20 amenities identified from training rows only; binary flags applied to both splits.
5. **Categorical encoding:** high-cardinality columns (>15 unique values) label-encoded with training-fit encoder; low-cardinality columns one-hot encoded with training structure reindexed onto test.
6. **Median imputation:** training medians applied to continuous columns; training mode applied to binary columns.
7. **Standard scaling:** `StandardScaler` fit on training continuous columns only; binary and one-hot columns excluded from scaling.

6.3 VIF Analysis

A Variance Inflation Factor check on the capacity cluster revealed:

Feature	VIF
Accommodates	13.18
Beds	9.44
Bedrooms	8.32
Bathrooms	7.95
Amenities Count	7.55
Host Verifications Count	6.86

Table 1: VIF scores for capacity-related features. `Accommodates` exceeds the $VIF = 10$ threshold and was dropped from feature sets used by linear and distance-based models.

7 Predictive Analysis

7.1 Task Definition

The primary predictive task is **binary demand classification**: predict whether a listing is *high demand* (label = 1) or *low demand* (label = 0) based on its features. The target is `demand_label` (median-split, 50/50 balanced). Ten classifiers were trained and evaluated on the held-out test set of 97,395 listings.

7.2 Feature Setup per Model Family

Model Family	Models	Accommodates Dropped?
Linear / Distance	LR, SVM, KNN, NB, MLP	Yes (VIF = 13.2)
Tree-based	RF, ET	No
Boosting	XGBoost, LightGBM, CatBoost	No

Table 2: Feature setup by model family. Scaling is applied globally via the post-processing pipeline (StandardScaler); binary/one-hot columns were excluded from scaling.

7.3 Results on Test Set

7.3.1 Logistic Regression

Class	Precision	Recall	F1-Score	Support
Low Demand	0.90	0.81	0.85	48,714
High Demand	0.83	0.91	0.87	48,681
Accuracy	0.86			
Macro Avg	0.86	0.86	0.86	97,395
Weighted Avg	0.86	0.86	0.86	97,395

Table 3: Logistic Regression — classification report on test set.

7.3.2 XGBoost

Class	Precision	Recall	F1-Score	Support
Low	0.94	0.91	0.92	48,714
High	0.91	0.94	0.92	48,681
Accuracy	0.9219			
Macro Avg	0.92	0.92	0.92	97,395
Weighted Avg	0.92	0.92	0.92	97,395

Table 4: XGBoost — classification report on test set. F1 (weighted) = 0.9218.

7.3.3 LightGBM

Class	Precision	Recall	F1-Score	Support
Low	0.94	0.90	0.92	48,714
High	0.91	0.94	0.92	48,681
Accuracy	0.9219			
Macro Avg	0.92	0.92	0.92	97,395
Weighted Avg	0.92	0.92	0.92	97,395

Table 5: LightGBM — classification report on test set. F1 (weighted) = 0.9218.

7.3.4 CatBoost

Class	Precision	Recall	F1-Score	Support
Low	0.93	0.90	0.92	48,714
High	0.91	0.94	0.92	48,681
Accuracy		0.9201		
Macro Avg	0.92	0.92	0.92	97,395
Weighted Avg	0.92	0.92	0.92	97,395

Table 6: CatBoost — classification report on test set. F1 (weighted) = 0.9201.

7.3.5 Naive Bayes

Class	Precision	Recall	F1-Score	Support
Low	0.81	0.22	0.34	48,714
High	0.55	0.95	0.69	48,681
Accuracy		0.5834		
Macro Avg	0.68	0.58	0.52	97,395
Weighted Avg	0.68	0.58	0.52	97,395

Table 7: Naive Bayes (Gaussian) — classification report on test set. F1 (weighted) = 0.5196.

7.3.6 MLP (Neural Network)

Class	Precision	Recall	F1-Score	Support
Low	0.91	0.90	0.91	48,714
High	0.90	0.92	0.91	48,681
Accuracy		0.9066		
Macro Avg	0.91	0.91	0.91	97,395
Weighted Avg	0.91	0.91	0.91	97,395

Table 8: MLP (Multi-Layer Perceptron) — classification report on test set. F1 (weighted) = 0.9066.

7.4 Model Comparison Summary

Model	Accuracy	F1 (W)	Prec (W)	Rec (W)	Rank
XGBoost	0.9219	0.9218	0.92	0.92	1
LightGBM	0.9219	0.9218	0.92	0.92	1
CatBoost	0.9201	0.9201	0.92	0.92	3
MLP	0.9066	0.9066	0.91	0.91	4
Logistic Reg.	0.8600	0.8600	0.86	0.86	5
Naive Bayes	0.5834	0.5196	0.68	0.58	6

Table 9: Model comparison on test set (97,395 listings). W = weighted average across both classes. RF, ET, KNN, and SVM results to be inserted upon completion.

7.5 Analysis of Results

Boosting models (XGBoost, LightGBM, CatBoost) achieve the highest accuracy (~92%), demonstrating that the demand signal is learnable from the available features. All three produce near-identical results, which is expected given they share the same gradient-boosted tree framework.

MLP achieves 90.7%, confirming that the relationships between features and demand are non-linear and that a deep representation helps even without explicit tree structure.

Logistic Regression achieves a solid 86% — a strong baseline for a linear model, indicating that approximately 86% of the variance in demand is linearly separable.

Naive Bayes underperforms significantly (58.3%), which is expected. GaussianNB assumes feature independence and Gaussian distributions — both violated here (features are correlated, many are skewed or binary). The recall asymmetry (0.22 for Low, 0.95 for High) indicates the model is biased towards predicting “High Demand” for nearly all listings.

Feature importance (from XGBoost): the top predictive features are `Days_Since_Last_Review` (0.175), `Host_Response_Time_Unknown` (0.124), and `Amenity_Shampoo` (0.054). The absence of any feature with importance >0.5 confirms there is no data leakage.

8 Results and Evaluation

8.1 Business Question Responses

Q1. What factors most influence the price of an Airbnb listing?

From EDA correlation analysis and boosting feature importances: **property size** (`Accommodates`, `Bedrooms`, `Bathrooms`) is the dominant price driver ($r > 0.45$ with log-price). **Room type** (Entire home >> Private >> Shared) creates discrete price tiers. **Location** (city, latitude/longitude) and **amenity count** are secondary factors.

Q2. Which locations and property types generate the highest occupancy rates?

High-demand listings concentrate in **New York, London, Paris, Los Angeles**,

and Barcelona. Entire homes in dense urban markets with flexible cancellation policies and high host response rates consistently achieve higher demand scores.

Q3. Can we predict listing price based on property features, location, and host characteristics?

Yes. The regression task (Price_log as target) was evaluated separately. The classification task demonstrated 92.2% accuracy in predicting demand category, confirming that listing features carry strong predictive signal for both price and occupancy.

Q4. What amenities and features are commonly associated with highly-rated listings?

From the amenity frequency analysis: **Wireless Internet, Kitchen, Heating, Essentials, Air Conditioning, Shampoo, and Hangers** are the most common amenities. Hosts with **verified identities, profile pictures, and flexible response times** correlate with higher demand, suggesting these trust signals matter beyond amenities alone.

Q5. How many listings are rented frequently to tourists vs. long-term residents?

The demand label distribution shows **approximately 50% of listings are high-demand** (frequent short-term rentals). Listings with Minimum Nights ≤ 3 and high Reviews per Month are characteristic of tourist-oriented rentals. Listings with Minimum Nights > 28 represent long-term or monthly rentals.

Q6. Which hosts are running a business with multiple listings and where are they located?

Approximately **30% of listings belong to multi-listing hosts** (Host Listings Count > 1). Professional hosts (5+ listings) are most concentrated in **New York, London, and Los Angeles**, consistent with these being the most commercially active markets.

9 Unsuccessful Trials

1. **Price_vs_city_median feature:** We initially engineered a feature dividing each listing's price by its city's median price to provide market-relative context. This was found to constitute **target leakage** for the regression task (the feature directly encodes the regression target's magnitude) and was removed.
2. **Global median for demand label:** The initial `label_engineering.py` computed the binary demand threshold using the global dataset median, which includes test rows. This was corrected in post-processing by recomputing the median from training rows only.
3. **Global tertiles for 3-class demand label:** Similarly, `pd.qcut` over all rows caused test row distributions to influence the tertile boundaries. Corrected by recomputing tertiles from training demand scores only.
4. **Scaling binary/one-hot columns:** An initial version of the post-processing pipeline passed all numeric columns (including binary amenity flags) through `StandardScaler`. This was corrected by excluding binary columns from scaling, as scaling binary features distorts their interpretation for distance-based models.

5. **Median imputation of label-encoded columns:** An initial imputation pass filled label-encoded integer columns with their median — meaningless for nominal codes. Corrected by applying median imputation to continuous columns only and mode imputation to binary columns.
6. **Naive Bayes on StandardScaler output:** `GaussianNB` receives negative values from `StandardScaler`, violating its non-negativity assumption. A `MinMaxScaler` rescaling was applied specifically for Naive Bayes to produce valid input.

10 Enhancements and Future Work

1. **Separate models per market:** A single global model conflates New York listings with rural Thai listings. Training market-specific models or including market-level normalization features could improve price prediction accuracy.
2. **NLP on free-text columns:** The dropped `Description`, `Summary`, and `House Rules` columns contain rich signal. Applying TF-IDF or sentence embeddings to these fields could meaningfully improve model performance.
3. **Temporal modelling:** The dataset is a point-in-time scrape. Incorporating time-series data (price over seasons, availability calendars) would allow demand forecasting rather than static demand classification.
4. **True occupancy data:** The demand proxy (reviews per month) is an imperfect substitute for actual booking data, which Airbnb does not publish. A model trained on verified booking records would produce more reliable demand predictions.
5. **Regression ensemble:** Combining the best classifiers (XGBoost + LightGBM + CatBoost) via stacking or voting could yield incremental accuracy improvements beyond the 92.2% ceiling reached by individual boosting models.
6. **Cloud deployment:** Migrating the pipeline to AWS EMR or Azure HDInsight for fully distributed Spark execution would reduce preprocessing time on the 1.94 GB dataset from minutes to seconds and enable real-time inference.

11 MapReduce Implementation

The K-Nearest Neighbours (KNN) algorithm was implemented from scratch using **Hadoop MapReduce in pseudo-distributed mode** (single machine, multiple processes) as required by the project specification.

11.1 Implementation Design

KNN classification requires computing the distance between a query point and all training points, then returning the majority class among the k nearest neighbours. This embarrassingly parallel task maps naturally to MapReduce:

- **Map phase:** Each mapper receives a subset of training points. For each query point, it computes the Euclidean distance to all training points in its partition and emits the top- k candidates as key-value pairs (`query_id`, (`distance`, `label`)).

- **Reduce phase:** The reducer receives all candidate (distance, label) pairs for a given query point, selects the global top- k by distance, and emits the majority-vote predicted class.

11.2 Configuration

- **Framework:** Apache Hadoop 3.x in pseudo-distributed mode
- **Language:** Python (via Hadoop Streaming)
- **Distance metric:** Euclidean distance on scaled features
- **Value of k :** tuned on validation set

A Repository Structure

```
project/
  data/
    airbnb-listings.csv      # raw data (1.94 GB)
    airbnb-cleaned.csv      # after preprocessing.
  py
    airbnb-labeled.csv      # after
  label_engineering.py
    splits/
      train.csv             # 80% training split
      test.csv              # 20% test split
    encoders/              # serialised sklearn
  artifacts
    standard_scaler.pkl
    train_medians.pkl
    clip_bounds.pkl
    ...
  notebooks/
    eda.ipynb
    logistic_regression.ipynb
    trees.ipynb             # RF + ET
    boosting.ipynb         # XGBoost + LightGBM +
  CatBoost
    svm_knn.ipynb
    nb_mlp.ipynb
  preprocessing.py
  label_engineering.py
  postprocessing.py
  mapreduce/
    mapper.py
    reducer.py
```